CSE234 Week 5: STL Set and Map

This week, we discuss the STL Set and Map containers. Sets are typically used as lookup containers to check if an element is part of the collection. Maps are used to store pairs of elements. One part of the pair acts as a key, while the other is the associated value.

Pairs

Before discussing sets and maps, let's discuss `std::pair`. `std::pair` is a struct that holds two elements, `first` and `second`. Pairs have two template parameters to specify the data types of its two parameters. The `std::make_pair()` function can be used to construct a pair:

```
// explicitly listing template arguments
std::pair<int, float> intFloat = std::make_pair<int, float>(2, 2.5);

// access members with .first and .second
assert(intFloat.first == 2);
assert(intFloat.second == 2.5);

// C++17 adds type deduction, so we don't have to specify
// the template arguments when it is obvious what they should be
std::pair intInt = std::make_pair(1, 2);

// be careful with strings
// this will be deduced as std::pair<int, const char*>
std::pair intCString = std::make_pair(1, "C String");

// if you want an std::string
// you need to specify or use the constructor
std::pair stringString = std::make_pair<std::string, std::string>("key
std::pair intStdString = std::make_pair(1, std::string("value"));

// initializer lists can also be used
std::pair<int, int> intInt2 = { 1, 2 };
```

Set (and Unordered Set)

The STL set is used store a collection of unique values. The multiset is used for a collection of values that are not unique, but do need to be kept in order. The multiset tracks the number of times a duplicate element has been "inserted". There are unordered variants of both the set and multiset. Most applications where sets are used do not need to preserve order, so `std::unordered_set` and `std::unordered_multiset` are typically preferred. The unordered versions offer much faster insertion, deletion, and access. Typically, the elements of a set are referred to as keys.

Set Template Parameters

Ordered sets (`std::set` and `std::multiset`) have 2 template parameters to consider. First, there is the data type of the elements being stored in the set. The second is the comparison function to determine the order of the elements. By default, it uses less than. This means that any type using the default comparison must implement the less than operator. If less than is not provided, a custom comparison class/struct needs to be provided. The struct must implement two functions: `bool comp()` and `bool equiv()`. Below is an example of declaring sets for integers, which have less than defined, and for a custom Item struct.

```
#include <set>
#include <string>

struct Item {
    std::string name;
    int id;
};

struct ItemComparator {
    // used in place of ==
    bool equiv(const Item& item1, const Item& item2) {
        return item1.id == item2.id;
    }
    // used in place of <
    bool comp(const Item& item1, const Item& item2) {
        return item1.id < item2.id;
    }
};

struct IntGreaterThanComparator {
    bool equiv(int x, int y) {
```

```
        return x == y;
    }
    bool comp(int x, int y) {
        return x < y;
    }
};

int main() {
    // int has a less than operator
    // therefore we don't need a comparison
    std::set<int> integers;

    // since the Item struct does not define operator<
    // a comparison struct needs to be implemented and provided
    std::set<Item, ItemComparator> items;

    // we can also provide a comparator for types that implement
    // less than if we want to use a custom comparison
    std::set<int, IntGreaterThanComparator> largeToSmallInts;

    return 0;
}
```

The unordered variants do not use a comparator. Instead, they require a hash. A hash is a function that returns a (sufficiently) unique integer for any input. Standard data types like int, float, std::string, etc. have hashes implemented for them, allowing the default hash class to be used. Creating custom hashes is beyond the scope of this course. You would need to implement a function object with a function `std::size_t hash()` that returns the hash of its input. We'll talk about function objects in week 7. Below is an example of declaring an `unordered_set` of strings:

```
#include <string>
#include <unordered_set>

int main() {
    std::unordered_set<std::string> strings;
    return 0;
}
```

Operations

All the set variants support `insert()`. For sets (ordered and unordered), `insert()` returns a pair that contains an iterator to the

inserted key, and a boolean that is true if the key was actually inserted. The value of `second` is false if the key is already present in the set. The multisets just return an iterator to the element regardless of if it was already present. An iterator can be passed as the first argument to specify where the new key is inserted. `insert()` can also be called with just the key to be inserted.

The `erase()` function is used to remove keys from the set. `erase()` can be passed either an iterator or a key. The `size()` function returns the current number of elements in the set. Despite the fact that `set` and `unordered_set` can only contain a single copy of a key, the `count()` function is implemented for all set variations. It is passed a key, and returns the number of occurrences of the key in the set. The `find()` function takes a key and returns an iterator to the location of the key in the set. If the key is not present, the `end()` iterator is returned. C++20 adds the `contains()` function, which takes a key and returns true if the set contains that key, and false if it does not. Before C++20, to check if an element was present, you'd compare the value of `find()` to the `end()` iterator or check if the value retuned by `count()` is 1 or 0.

Example Applications

Sets can be used anytime you want to create a collection of unique items. Sets offer faster retrieval of items than a list, with the penalty of slower insertion and deletion than list. Since vectors have random access, they offer faster retrieval of items (although searching is slower), but inserting items that are not at the end of the vector is slower than inserting an item to a set.

Unordered sets are faster than list and vector in almost all cases, but do not preserve order. If you need to store a collection of items where order is not needed, unordered sets are best. Typically, unordered sets are used when you need to create a "lookup table" to check whether an item is part of a collection.

Map (and Unordered Map)

STL Maps are used to store (key, value) pairs. There are four variations of map: `std::map`, `std::multimap`, `std::unordered_map` and

`std::unordered_multimap`. If the keys in the map must be unique, `std::map` or `std::unordered_map` can be used. If there can be duplicate keys, then `std::multimap` and `std::unordered_multimap` must be used. The `std::map` and `std::multimap` collections preserve the insertion order of elements. If this is not required, using the `unordered` variants provide a performance boost. The ordered versions are rarely needed. The multimap variations are also not as useful, as the value of the map can just be a list or vector in that case. The map types are basically the same as the corresponding set, except the keys now have an associated value. Maps are similar to dictionaries in Python, objects in JavaScript, and associative arrays in PHP. Maps allow data formats like JSON to be represented in C++.

Map Template Parameters

Map template parameters are the same as they are for set, with the addition of the value's template parameters. In an ordered map, the parameters are in the order `<key, value, comparator>`, while for unordered maps they are `<key, value, hash>`.

Operations

Pretty much all the same methods as sets apply to map as well. For `insert()`, an `std::pair` with the key and value must be passed, rather than a single key. `erase()` only expects a key, and cannot delete elements based on their value. `count()` and `find()` also operate based on the keys, not the values.
In addition to the shared behavior with sets, maps and unordered maps (not multimaps, though) also support random access using the subscript `operator[]` and `at()` member function. Passing a key as the index allows the corresponding value to be accessed. Unlike with vector, where the subscript operator could not be used to insert an element, it can be used for insertions with maps. For a normal map (not a multimap), if the key already exists in the map, its value will be overwritten by inserting with the subscript. The `insert()` function will not overwrite the value, and will instead return a pair that contains an iterator to that element as its `first` value, and false as `second`.

Example Applications

Maps are useful when you have some kind of ID to associate with data items. It makes finding a specific data item in a collection fast, especially if the unordered variant is used.

Example

Here, we'll implement a rudimentary ordering system using a map. Since the order of the elements is not important for this application (we are only looking up item numbers), an `unordered_map` will be used. A second map will be used to keep track of the items added to the order. Note that I am using a try-catch here to handle invalid item number entries. The details of how to use try-catches will be discussed next week. For now, the usage here should be simple enough that you can follow.

```
#include <iostream>
#include <string>
#include <unordered_map>

struct Item {
    double price;
    std::string name;
    int id;
    // overloading output operator
    friend std::ostream& operator<<(std::ostream& out, const Item& ite
        out << "Item ID:   " << item.id << "\n";
        out << "Item Name: " << item.name << "\n";
        out << "Price:    $" << item.price << "\n";

        return out;
    }
};

void printCart(
    const std::unordered_map<int, int>& cart,
    const std::unordered_map<int, Item>& items
    ) {

    double total = 0;
    // a range based for loop will iterate through the
    // {key, value} std::pairs in the map
    for (const auto item: cart) {
        double price = items.at(item.first).price * item.second;
        total += price;
        std::cout << items.at(item.first).name << " x "
            << item.second << " --- $" << price << "\n\n";
    }
```

```
    std::cout << "\nTotal: $" << total << "\n";
}

int main() {
    // creating map of valid items
    // using intializer lists to fill map
    std::unordered_map<int, Item> items = {
        {1000, {1.54, "item1", 1000}},
        {1012, {2.63, "item2", 1012}},
        {1075, {6.25, "item3", 1075}},
        {1134, {7.45, "item4", 1134}},
        {1759, {6.73, "item5", 1759}}
    };

    // map to hold cart. key is the item number and value is quantity
    std::unordered_map<int, int> cart;

    // there is an if with a break statement to end the loop later
    while (true) {
        int itemNumber;
        int quantity;
        try {
            std::cout << "Enter the item number: ";
            std::cin >> itemNumber;
            // at will throw an std::out_of_range error if
            // the item is not in the map
            Item item = items.at(itemNumber);
            // if it is, the code in the try continues
            std::cout << "You selected:\n";
            std::cout << item;
            std::cout << "Enter quantity: ";
            std::cin >> quantity;

            // if the item is not in the cart, the .second of
            // insert() will be true
            if (cart.insert({itemNumber, quantity}).second) {
                std::cout << "Item added to cart\n";
            }
            // otherwise, the item is already in the cart
            // instead, ask if the user wants to add more
            else {
                std::cout << "Item already in cart\n";
                std::cout << "Add " << quantity << " more [y/n]? ";
                char response;
                std::cin >> response;
                if (response == 'y' || response == 'Y') {
                    cart[itemNumber] += quantity;
                }
            }
        }
```

```
        // the catch will run if .at() throws an exception
        // this error message will be printed
        catch (std::out_of_range& ex) {
            std::cerr << "Invalid item number\n";
        }
        // this is after the try catch, so it runs no matter what
        std::cout << "Continue adding items [y/n]? ";
        char response;
        std::cin >> response;
        if (response == 'n' || response == 'N') {
            break;
        }
    }

    std::cout << "\n\n\n";
    printCart(cart, items);

    return 0;
}
```

Example Output:

```
Enter the item number: 1000
You selected:
Item ID:   1000
Item Name: item1
Price:    $1.54
Enter quantity: 5
Item added to cart
Continue adding items [y/n]? y
Enter the item number: 1000
You selected:
Item ID:   1000
Item Name: item1
Price:    $1.54
Enter quantity: 2
Item already in cart
Add 2 more [y/n]? y
Continue adding items [y/n]? y
Enter the item number: 1167
Invalid item number
Continue adding items [y/n]? y
Enter the item number: 1759
You selected:
Item ID:   1759
Item Name: item5
Price:    $6.73
Enter quantity: 8
Item added to cart
Continue adding items [y/n]? n
```

```
item5 x 8 --- $53.84

item1 x 7 --- $10.78


Total: $64.62
```

Conclusion

This week, we discussed sets and maps. The C++ STL implements ordered and unordered variants of sets and maps. The unordered versions are more commonly used, since they are much faster. The elements of a set are typically referred to as keys. Sets are used to create collections that we later check if an element is a part of. Multisets count the number of times an element is entered into the set. If the keys need to have a value associated with them, a map can be used. Maps store {key, value} pairs, and support access via the subscript operator and `at()` member function.
